Introduction to R & RStudio

Data Science for the Psychological Sciences

What is R?

R is a free, open-source programming environment designed for statistical computing, data analysis, and scientific visualization. It combines an interactive console for exploratory work with a rich ecosystem of packages that support everything from data cleaning and modeling to reproducible reporting and interactive applications. In the R environment, users typically write scripts that import and transform data, generate graphics, and fit statistical or machine learning models, with results that can be documented and shared through tools such as R Markdown and Quarto. Because R is widely used in research and industry and is especially strong for statistical methodology and high-quality plotting, it is a practical platform for psychologists and other social scientists who need transparent, replicable workflows for analyzing data and communicating findings.

Unit goals:

1. Introduce the RStudio integrated development environment (IDE).
2. To provide a hands-on introduction to the R programming environment.

RStudio/Rmarkdown

R is a standalone application, meaning that it can be installed and used as-is, but both learning and using R can be facilitated by combining it with an integrated development environment (IDE) like Jupyter Notebooks, Rcommander, or Rstudio (just to name a few).

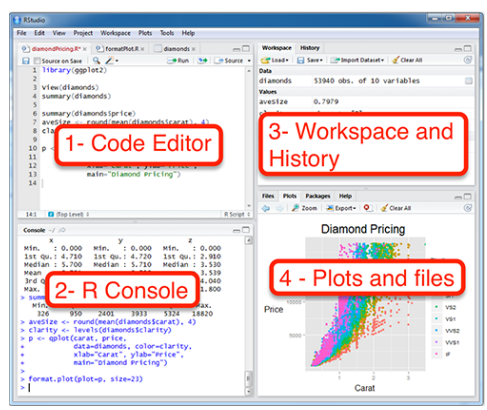

Figure 1: RStudio interface.

In this class, you will be using RStudio, an integrated development environment (IDE) for the R programming language that makes it easier to write code, manage files, and run analyses in one place. RStudio provides a clear interface for working with scripts, viewing data, creating plots, and installing and using packages, which supports an efficient and reproducible workflow. Over the course of the term, you will use RStudio to import and clean data, generate visualizations, and conduct statistical and machine learning analyses, while documenting your work in a way that can be shared and revisited (see Figure 1).

An advantage of using RStudio is that it permits the seamless integration of text, code, and output within a single, coherent workflow, which is especially valuable for producing transparent and reproducible analyses. By combining narrative explanation with executable R code, tables, and figures, you can document not

only what results you obtained, but also exactly how you obtained them—making your work easier to verify, update, and share. This approach supports best practices in modern data science and psychological research, where the goal is to move from ad hoc analyses to well-organized projects that can be rerun end-to-end as data or research questions evolve.

R Markdown Formatting Basics

RStudio supports writing and analysis through R Markdown, a format that lets you combine plain-language text with executable R code in a single document that can be rendered into polished outputs such as HTML, PDF, or Word. This "literate programming" approach is valuable because it keeps your narrative, analysis, and results together, reducing the risk of copy-and-paste errors and making it straightforward to reproduce your work later. In practice, R Markdown encourages a clean, transparent workflow: you write what you are doing and why, run the code that performs the analysis, and generate figures and tables directly from the underlying data—so the final document can be regenerated whenever inputs change, rather than manually rebuilt. R Markdown allows you to create beautifully formatted documents, uses a simple syntax to create headers, lists, links, and more.

R Code Chunks

As you'll see throughout this document, R code is placed in "chunks," which are clearly marked sections of an R Markdown file that tell RStudio what to run and what to display in the final output. Each chunk can contain one or more lines of code, and when you knit the document, R executes the code and inserts the resulting tables, figures, or printed output directly into the report. This chunk-based structure helps keep analyses organized, makes it easier to rerun and troubleshoot specific steps, and supports reproducibility by ensuring that the results shown in the document are generated from the code that accompanies them.

You can start a code chunk in an Rmarkdown document using three accent characters followed by the lowercase letter r in braces and you can end the code chunk using another set of three accent characters (see example below). Any R code in between the opening and closing of the code chunk is executable. Any output from the code execution will appear directly below the chunk in the final, knitted document.

```{r}
R code goes here
```

The header at the beginning of each chunk, the `{r}`, can include chunk options/parameters that control how the code is run and how its output appears in the final document. These options let you decide whether a chunk should be executed at all (`eval`), whether the code itself should be shown to the reader (`echo`), and whether messages or warnings should be displayed (`message`, `warning`). You can also control how figures are produced and formatted—for example, providing a figure caption (`fig.cap`), adjusting figure size (`fig.width`, `fig.height`), setting the output width (`out.width`), or specifying where figures should be placed in a PDF (`fig.align`). Taken together, chunk options are a practical tool for creating documents that are both readable and reproducible: you can keep the analysis code that generates results while tailoring what the audience sees and how polished the output looks.

Headers

Headers (also called headings) are used to organize your document into clear sections and subsections, making it easier to read and navigate. Headers are created with one or more # symbols at the start of a line:

a single # typically marks a main section title, ## creates a subsection, and ### creates a sub-subsection, with additional # symbols indicating deeper levels. Beyond improving readability, headers play a practical role when you render the document: they can automatically populate a table of contents, enable section-based navigation in HTML output, and provide a consistent structure for longer reports, assignments, or manuscripts. Consistent heading levels also help you communicate the flow of your analysis—from research question, to methods, to results, to interpretation—so the narrative and code remain logically connected.

```
#   Level 1 Header
##  Leavel 2 Header
### Header 3
```

Text Emphasis

You can make text *italic* or **bold**.

- *Italic text* is created with `*asterisks*` or `_underscores_`.
- **Bold text** is created with `**double asterisks**` or `__double underscores__`.
- You can even do ***bold and italic*** with `***triple asterisks***`.

Lists

Lists are a simple way to organize information in R Markdown, whether you are summarizing key points, outlining steps in an analysis, or presenting results clearly. Use an unordered (bulleted) list when the order of items does not matter, and an ordered (numbered) list when you want to emphasize sequence, ranking, or procedure. You can also create nested lists by indenting sub-items beneath a main bullet or number, which is useful for adding detail without cluttering the main flow of the document.

Unordered List:

```
* Item A
* Item B
  * Sub-item B1
  * Sub-item B2
```

Ordered List:

```
1. First item
2. Second item
3. Third item
```

Links and Images

Links and images are added using straightforward Markdown syntax and are useful for connecting your document to external resources (e.g., documentation, datasets, articles) and for including figures, screenshots, or diagrams that support your narrative.

Links: Create a hyperlink with `[display text](URL)`. For example: The R Project. Be sure to choose display text that clearly describes where the link goes (e.g., "RStudio Cheatsheet" rather than "click here"), especially in instructional materials.

Images: Embed an image with `![alt text](path/to/image.png)`. The alt text is a brief description of the image that improves accessibility and appears if the image cannot be loaded. Image paths are

usually relative to your project folder (for example, images/rstudio.png), which helps keep your document portable and easier to share.

If you need to control image size or placement, you can add simple formatting. For example, you can set a width directly on the Markdown image using:

```
![RStudio interface](rstudio.png){width=200px}
```

Blockquotes and Inline Code

Sometimes you also want to refer to code elements directly in your writing—for example, the name of a dataset, an object you created, or a function you are asking students to use. In those cases, inline code helps the reader immediately recognize that the text refers to code. To format inline code, wrap it in backticks. For example, `` `code` `` will print as `code`. Inline code is especially helpful for clarifying small but important details such as distinguishing between mean (an object name) and mean() (a function), or showing argument names like `na.rm = TRUE` exactly as it should be typed.

> Blockquotes (like this paragraph) are a simple way to visually separate quoted material or "callout" text from the rest of your document. To create a blockquote, start a line with > and R Markdown will indent the text and format it differently from the main body. This is useful for including short quotations, emphasizing definitions, or adding instructor notes and reminders. You can also create multi-line blockquotes by placing > at the beginning of each line.
>
> > You can also use >> to nest blockquotes (or include lists inside them) when you need more structure.

The YAML Header

At the very top of an R Markdown file, you will need a block of text surrounded by three dashes:

```
---
title: "My Document Title"
author: "Your Name"
date: "2025-12-29"
output: html_document
---
```

This section is called the YAML header. It controls the overall settings for your document, including the output format, appearance, and optional features such as a table of contents.

Common YAML fields:

- title: The document title displayed at the top of the rendered output
- author: Optional author line
- date: Optional date line (you can remove it)
- output: The format R Markdown will generate (HTML, PDF, Word, etc.)

If you do not want the author/date to appear, delete those lines or set them to blank (empty quote)

Choosing Output Formats and Options

1. ***HTML output (most flexible)***

```
---
title: "Lab Report"
output:
html_document:
`toc: true`
`toc_depth: 2`
`number_sections: true`
`theme: readable`
---
```

What these options do:

- toc: true adds a table of contents
- toc_depth controls how many header levels appear in the TOC
- number_sections automatically numbers headers
- theme changes the visual style of the HTML output

2. ***PDF output (print-friendly, more strict formatting)***

```
---
title: "Lab Report"
output:
pdf_document:
toc: true
number_sections: true
---
```

Note: PDF output requires a LaTeX installation. If you get LaTeX errors, you may need to install TinyTeX.

3. ***Word output (useful for submissions)***

```
---
title: "Lab Report"
output:
word_document:
toc: true
---
```

Introduction to R

R's real power comes from the availability of an expansive list of packages that extend R's capabilities.

Installing and Loading Packages

Regardless of how you choose to use R, you will start with only the base R package. Analogous to "modules" in Python, the R base program can be extended by adding "packages". Packages are collections of scripts and functions usually suited to a particular analysis or task. The core R library includes many pre-loaded packages by default. To get a list of all currently installed packages, you can use the `library()` function without any input arguments.

Loading a Package

Note that some libraries are installed with the R base package, but they may not be loaded into the default R environment. For example, the library called "foreign" includes functions that will allow users to read/write data in formats used by other (i.e., foreign) software (e.g., SPSS, SAS, etc.). Pre-installed libraries can be loaded for use in the current R session using the `library()` function, with the name of the library as the input argument.

The example below illustrates how you would load the `dplyr` package.

```
library(dplyr)
```

Installing a New Package

In the event that a library is not already installed, it can usually be added to an existing R installation using the `install.packages()` function. For example, in a typical R installation, the command: `install.packages("foreign")` will automatically download and install the "foreign" package so that it can be loaded when you want to use it.

You only need to **install** a package once, but you need to **load** it every time you start a new R session.

```
#install.packages("dplyr")
#library("dplyr")
```

R as a Calculator: Mathematical Operators

Before we start using the scripts and functions contained in some of R's packages, let's take a moment to get familiar with the R environment itself. At its simplest, R can be used as a powerful calculator: you can type arithmetic expressions directly into the Console and R will evaluate them immediately. This may seem basic, but it is a useful way to learn how R reads input, returns output, and represents numbers. These are foundational skills that carry over when you begin writing longer scripts, creating objects, and using functions for data analysis.

```
# Addition
5 + 3
```

```
## [1] 8
```

```
# Subtraction
10 - 4
```

```
## [1] 6
```

```r
# Multiplication
6 * 7
```

```
## [1] 42
```

```r
# Division
20 / 4
```

```
## [1] 5
```

```r
# Exponentiation (raising to a power)
23  # 2 cubed
```

```
## [1] 8
```

```r
# Modulus (finds the remainder of a division)
10 %% 3 # Remainder of 10 divided by 3 is 1
```

```
## [1] 1
```

Notice that R prints the result immediately after each calculation. In knitted R Markdown output, these results are typically prefixed with ##, and each printed line may be tagged with an index such as `[1]`. In the examples above, some lines begin with the # symbol. This is the comment character: anything following # on a line is treated as a note to the reader and is ignored by R when the code runs.

Variables in R

A variable in R is a name that you assign to a value so you can use it later. Instead of repeatedly typing the same number, text, or dataset, you store it in an object and refer to it by its name. This is one of the most important ideas in R because nearly everything you do like running analyses, creating plots, and working with data depends on creating and reusing objects.

Creating variables with `<-` (assignment)

To create a variable, you use the assignment operator `<-` (you will also see = used, but `<-` is the most common style in R):

For example, typing `age <- 52` will assign the value 52 to a variable named "age". Likewise, typing `weight <- 240` will assign the value 240 to the variable named "weight".

After a value has been assignd to a variable, R stores the value in memory. If you type the variable name, R prints its contents. For example

```r
age <- 52
```

```r
age
```

```
## [1] 52
```

When creating variables, there are some general rules and best practices that you should follow.

- Variable names should be descriptive and easy to read.
- Names can include
  - letters

    - numbers
    - Underscores, _
    - Periods, .
- Names cannot start with a number
- R is case-sensitive (Age and age are different objects)

Good examples:

```
total_score <- 18
mean_rt <- 542
```

Variables are not limited to numbers. They can store text, logical values, or more complex objects like vectors, data frames, etc. (more on these below):

Here is an example of a text variable.

```
name <- "Jordan"
```

Here is an example of a logical (TRUE/FALSE) variable.

```
is_complete <- TRUE
```

Just about **any** R object can be stored as a variable.

You can always check what objects you have created in the Environment pane in RStudio, which lists your variables and lets you inspect their values. As you progress, you will begin creating variables that store not only single values, but also datasets, models, and plots—making variables the building blocks of nearly everything you do in R.

Relational and Logical Operators

Relational Operators

Relational operators compare two values and return a logical result: TRUE or FALSE. These logical values are foundational in R because they are what drive decision-making in code (e.g., if statements) and filtering data (e.g., selecting rows that meet a condition). When you read an expression like `x > y`, you can interpret it as a question like "Is x greater than y?", and R answers with TRUE or FALSE.

A few notes to keep in mind:

- Use == for comparison; a single = is used for assignment in some contexts.
- Comparisons are type-sensitive: comparing numbers to text will not behave the same as comparing two numbers.
- == checks exact equality, which can be tricky with some decimal values due to rounding; later you'll learn safer approaches for comparing floating-point numbers.

Relational Operators in R

- == : Equal to
- ! = : Not equal to
- < : Less than
- > : Greater than
- <= : Less than or equal to
- >= : Greater than or equal to

Below are several examples of logical operators and their interpretation:

```
#Create variables x and y and assign them values
x <- 10
y <- 5

# Is x equal to 10?
x == 10
```

```
## [1] TRUE
```

```
# Is y not equal to 5?
y != 5
```

```
## [1] FALSE
```

```
# Is x greater than y?
x > y
```

```
## [1] TRUE
```

Logical Operators

Logical operators allow you to combine or modify logical statements (expressions that evaluate to TRUE or FALSE). They are essential for making decisions in code (e.g., `if` statements) and for selecting data that meet multiple criteria (e.g., filtering rows where several conditions are satisfied). In practice, you will often use logical operators to build "rules," such as "include participants who are older than 18 and completed the study" or "flag cases where a score is unusually high or unusually low."

The three core logical operators

- `&` : AND (returns TRUE only when both conditions are TRUE.)
- `|` : OR (returns TRUE when at least one condition is TRUE.)
- `!` : NOT (reverses the result of a logical statement (TRUE becomes FALSE, and vice versa))

A useful way to read these is as plain-language questions:

(x > 5) & (y < 10) $\longrightarrow$ "Is x greater than 5 and y less than 10?"

(x == 5) | (y == 5) $\longrightarrow$ "Is x equal to 5 or y equal to 5?"

!(x > y) $\longrightarrow$ "Is it not true that x is greater than y?"

Why the parentheses matter:

Notice the parentheses around each comparison, like (x > 5) and (y < 10). Each comparison produces a TRUE/FALSE value, and then & or | combines those results. While R can sometimes interpret expressions without parentheses, using them makes your code easier to read and reduces mistakes.

R has two versions of AND/OR:

& and | are vectorized, meaning they compare element-by-element when you are working with vectors (this is what you usually want when filtering data, we will get to that later).

&& and || evaluate only the first element and are commonly used inside if statements when you truly want a single TRUE/FALSE.

For most introductory work (and especially data filtering), & and | are the correct "default. Several examples are illustrated below.

```
# Is x greater than 5 AND y less than 10?
(x > 5) & (y < 10)
```

```
## [1] TRUE
```

```
# Is x equal to 5 OR y equal to 5?
(x == 5) | (y == 5)
```

```
## [1] TRUE
```

```
# Is x NOT greater than y?
!(x > y)
```

```
## [1] FALSE
```

Data Types

In R, you can store values in **variables** so you can use them later. The assignment operator is `<-`.

Atomic Data Types

In R, the simplest values you work with are built from *atomic* data types, which represent the basic kinds of information R can store. The most common atomic types are **logical** (TRUE/FALSE), **integer** (whole numbers), **double/numeric** (real numbers with decimals), **character** (text strings), and **complex** (numbers with real and imaginary parts, used less often in typical data analysis).

```
# Integer: For whole numbers
my_number <- 42

# Numeric: For numbers (real numbers with decimals)
another_number <- 3.14

# Character: For text (strings), always in quotes
my_text <- "Hello, R!"

# Logical: For TRUE or FALSE values
is_learning <- TRUE

# We can print the variables to see their values
print(my_number)
```

```
## [1] 42
```

```
print(my_text)
```

```
## [1] "Hello, R!"
```

```
print(is_learning)
```

```
## [1] TRUE
```

In R, NA and NaN are special atomic values used to represent missingness and undefined results, and they always belong to one of R's atomic vector types (logical, integer, double/numeric, character, complex, raw). NA means "missing value"—the data should exist conceptually, but it is not available. Importantly, NA is typed: you can have NA for different atomic types (e.g., NA_integer_, NA_real_, NA_character_, NA_complex_).

NaN means "Not a Number"—it is a numeric value that represents an undefined or unrepresentable mathematical result (most commonly 0/0). Unlike NA, which comes from missing data, NaN comes from invalid arithmetic. NaN is also treated as missing in many contexts: for example, is.na(NaN) returns TRUE, but is.nan(NA) returns FALSE. In short: NA = missing data (typed, can occur in any atomic type), whereas NaN = an undefined numeric result (only numeric, typically from computations).

Vectors

A vector is a one-dimensional collection of values (elements) that all share the same data type, such as numeric values, character strings, or logical (TRUE/FALSE) values. You can create a vector using the `c()` function (short for "combine"), which joins individual elements into a single object. For example, `c(1, 2, 3)` creates a numeric vector and `c("A", "B", "C")` creates a character vector. Because vectors are homogeneous, if you mix types—such as numbers and text, R will automatically convert the entire vector to a common type (often character), which is useful to know when preparing and cleaning data.

```r
# A numeric vector
numeric_vector <- c(1, 10, 50, 100)

# A character vector
character_vector <- c("apple", "banana", "cherry")

print(numeric_vector)
```

```
## [1]   1  10  50 100
```

```r
print(character_vector)
```

```
## [1] "apple"  "banana" "cherry"
```

Data Frames

The most common way to store tabular data in R is in a **data frame**. You can think of a data frame like a spreadsheet: rows represent observations (e.g., participants or trials) and columns represent variables (e.g., age, condition, reaction time). Unlike a single vector, each column in a data frame can have a different data type (numeric, character, logical, etc.), which makes data frames well-suited for real datasets. Most of the data manipulation and analysis you will do in this course, like filtering cases, computing new variables, and fitting models, will begin with a data frame.

Creating a Data Frame

```r
# Create a few vectors
student_id <- c(1, 2, 3, 4)
student_name <- c("Alice", "Bob", "Charlie", "David")
test_score <- c(88, 92, 75, 85)
```

```r
# Combine them into a data frame
student_data <- data.frame(
  ID = student_id,
  Name = student_name,
  Score = test_score
)

# Print the data frame
print(student_data)
```

```
##   ID    Name Score
## 1  1   Alice    88
## 2  2     Bob    92
## 3  3 Charlie    75
## 4  4   David    85
```

Inspecting a Data Frame

There are several useful functions to get a quick overview of your data before you begin cleaning, visualizing, or analyzing it. These "inspection" steps help you confirm that the dataset loaded correctly, understand what each variable represents, and catch common issues early (for example, variables imported as the wrong type, unexpected missing values, or values that fall outside a plausible range). In general, you will use `head()` and `tail()` to spot-check the first and last few rows, `str()` to see the names and data types of each column (and how many observations you have), and `summary()` to obtain quick descriptive information—such as minima and maxima for numeric variables or frequency counts for categorical variables. Spending a minute on these checks can prevent much larger problems later in your workflow.

```r
# head() shows the first few rows
head(student_data)
```

```
##   ID    Name Score
## 1  1   Alice    88
## 2  2     Bob    92
## 3  3 Charlie    75
## 4  4   David    85
```

```r
# tail() shows the last few rows
tail(student_data)
```

```
##   ID    Name Score
## 1  1   Alice    88
## 2  2     Bob    92
## 3  3 Charlie    75
## 4  4   David    85
```

```r
# str() shows the structure of the data frame (str-ucture)
str(student_data)
```

```
## 'data.frame':    4 obs. of  3 variables:
##  $ ID   : num  1 2 3 4
```

```
##  $ Name : chr  "Alice" "Bob" "Charlie" "David"
##  $ Score: num  88 92 75 85
```

```r
# summary() provides summary statistics for each column
summary(student_data)
```

```
##        ID           Name               Score
##  Min.   :1.00   Length:4           Min.   :75.0
##  1st Qu.:1.75   Class :character   1st Qu.:82.5
##  Median :2.50   Mode  :character   Median :86.5
##  Mean   :2.50                      Mean   :85.0
##  3rd Qu.:3.25                      3rd Qu.:89.0
##  Max.   :4.00                      Max.   :92.0
```

Indexing and Slicing: Accessing Your Data

Once you have data stored in a vector or a data frame, you need a way to access specific parts of it. This process is called **indexing** (accessing elements) or **slicing** (accessing a sequence of elements). R is a **1-indexed** language, which means the first element is at position 1, the second at position 2, and so on.

Indexing Vectors

We use square brackets `[]` to access elements of a vector.

```r
# Let's create a vector to work with
ages <- c(25, 31, 19, 42, 50, 22)
```

1. Accessing a Single Element by Position To get the element at a specific position, place the position number in the brackets.

```r
# Get the third element
ages[3]
```

```
## [1] 19
```

2. Accessing Multiple Elements by Position You can pass a vector of positions into the brackets to get multiple elements.

```r
# Get the first and fifth elements
ages[c(1, 5)]
```

```
## [1] 25 50
```

3. Accessing a Sequence of Elements Use the colon operator `:` to get a continuous slice of the vector.

```r
# Get all elements from the second to the fourth
ages[2:4]
```

```
## [1] 31 19 42
```

```r
# Get all elements from the first to the last, by two.
ages[seq(1,length(ages),by=2)]
```

```
## [1] 25 19 50
```

Note that the example above uses the `seq()` function, which produces a sequence of numbers that we can use as index positions. The first input argument, 1, tells `seq()` to start the sequence at 1 (the first element of the output vector). The second argument, `length(ages)`, tells the function to continue up to the length of the vector (i.e., the last valid index). Finally, `by = 2` tells `seq()` to step by two each time, producing the sequence 1, 3, 5, 7, and so on. When you use that sequence inside brackets, R returns the elements in those positions, effectively selecting every other element starting with the first.

4. Logical Indexing This is a very powerful technique. You can use a logical vector (TRUE/FALSE values) to select only the elements corresponding to a TRUE value. This is a common technique used for filtering data.

Recall that the logical operators are vectorized, meaning they operate element-by-element across a vector and return a logical result for each element. For example, because `ages` is a vector in our example above, the expression `ages > 30` produces a logical vector of the same length as ages, with TRUE wherever the condition is met and FALSE elsewhere. When you place that logical vector inside brackets, R keeps only the elements that correspond to TRUE values and drops the rest. This approach scales naturally from vectors to data frames, where the same idea is used to keep only rows that satisfy a condition (e.g., selecting participants above a certain age or trials from a particular condition).

```r
# Which ages are over 30?
ages > 30
```

```
## [1] FALSE  TRUE FALSE  TRUE  TRUE FALSE
```

```r
# Now use that logical vector to select ONLY the ages over 30
ages[ages > 30]
```

```
## [1] 31 42 50
```

5. Excluding Elements Use a negative number to exclude the element at that position.

```r
# Get all ages EXCEPT the second one
ages[-2]
```

```
## [1] 25 19 42 50 22
```

Indexing Data Frames

Data frames have two dimensions: rows and columns. We still use square brackets `[]`, but the notation is `[rows, columns]`. The comma is crucial!

```r
# Let's create a data frame to work with
student_data <- data.frame(
  ID = c(101, 102, 103, 104),
  Name = c("Alice", "Bob", "Charlie", "David"),
  Score = c(88, 92, 75, 85),
  Major = c("Biology", "History", "Math", "Biology")
)
print(student_data)
```

```
##    ID    Name Score   Major
```

```
## 1 101   Alice    88 Biology
## 2 102     Bob    92 History
## 3 103 Charlie    75    Math
## 4 104   David    85 Biology
```

1. The $ Operator (Most Common) The easiest and most common way to select an entire column by its name is with the $ operator.

```r
# Get the entire 'Score' column
student_data$Score
```

```
## [1] 88 92 75 85
```

```r
# Calculate the mean of the 'Score' column
mean(student_data$Score)
```

```
## [1] 85
```

2. Using `[rows, columns]` Notation

- To select **rows**, specify the row number(s) *before* the comma.
- To select **columns**, specify the column number(s) or name(s) *after* the comma.
- Leave one side blank to select *all* rows or *all* columns.

```r
# Get the entire third row
student_data[3, ]
```

```
##    ID    Name Score Major
## 3 103 Charlie    75  Math
```

```r
# Get the first and third rows
student_data[c(1, 3), ]
```

```
##    ID    Name Score   Major
## 1 101   Alice    88 Biology
## 3 103 Charlie    75    Math
```

```r
# Get the entire 'Name' column (the 2nd column)
# Note: This returns a vector
student_data[, 2]
```

```
## [1] "Alice"   "Bob"     "Charlie" "David"
```

```r
# Get a specific cell: the name of the student in the 3rd row
student_data[3, 2]
```

```
## [1] "Charlie"
```

```r
# Get multiple columns by name
student_data[, c("Name", "Major")]
```

```
##      Name   Major
## 1   Alice Biology
## 2     Bob History
```

```
## 3 Charlie    Math
## 4   David Biology
```

3. Filtering Data Frames (Very Important!) This combines the logical indexing we saw with vectors and the `[rows, columns]` notation. We create a logical condition to select which rows we want to keep.

```r
# Create a logical vector: Which students scored above 85?
student_data$Score > 85
```

```
## [1]  TRUE  TRUE FALSE FALSE
```

```r
# Now, use that logical vector in the 'rows' position to filter the data frame
high_scorers <- student_data[student_data$Score > 85, ]
print(high_scorers)
```

```
##    ID  Name Score   Major
## 1 101 Alice    88 Biology
## 2 102   Bob    92 History
```

```r
# You can also filter by text
# Find all the Biology majors
biology_majors <- student_data[student_data$Major == "Biology", ]
print(biology_majors)
```

```
##    ID  Name Score   Major
## 1 101 Alice    88 Biology
## 4 104 David    85 Biology
```

Using Built-in Functions and Inspecting R Objects

One of the most important ideas in R is that many of the "commands" you type are actually **functions**. A function is a special object that performs an operation when you call it using parentheses, for example `mean(x)` or `head(data)`. Functions often take arguments (inputs) and return a value (output).

Even in the first few pages of this course, you have already been using many functions, including:

- Creating and combining values: `c()`, `data.frame()`, `seq()`, `length()`
- Printing and previewing: `print()`, `head()`, `tail()`
- Summarizing and inspecting: `summary()`, `str()`
- Working with packages: `install.packages()`, `library()`

A useful habit is to notice the parentheses: if you see something like `head(student_data)`, you are calling a function named `head` on the object `student_data`.

Note that many functions have sensible defaults, but you can often override them by providing arguments. For example, `head()` shows the first 6 rows by default, but you can ask for a different number using the `n` argument:

```r
head(student_data) # default: 6 rows
```

```
##    ID    Name Score   Major
## 1 101   Alice    88 Biology
## 2 102     Bob    92 History
```

```
## 3 103 Charlie    75    Math
## 4 104   David    85 Biology
```

```r
head(student_data, n = 2) # explicitly request 2 rows
```

```
##    ID  Name Score   Major
## 1 101 Alice    88 Biology
## 2 102   Bob    92 History
```

Checking an object's "type" in R

In this chapter, we have covered a number of different types of R objects. One very useful function for checking the atomic "type" of an object is to use the `typeof(x)` function, which reports the object's underlying storage type (e.g., "double", "integer", "character", etc.)

Higher-level objects in R (e.g., data frames, matrices, and other structured containers) are typically built on top of simpler underlying types, but they behave differently because R assigns them a particular "class". The function `class(x)` reports that class, which tells you how R will generally treat the object in common operations and in many "generic" functions (for example, how it prints, how subsetting behaves, and what you get when you run functions like `summary()` or `plot()`). For instance, a data frame is internally a special kind of list, but because its class is "data.frame", R handles it as a rectangular table with rows and columns. This becomes even more important once you begin using modern data tools (e.g., tibbles) or fitting statistical models, because these objects rely on class to ensure that the right methods are used for inspection, visualization, and reporting.

The function `str(x)` (short for "structure") gives a compact, human-readable summary of an object that is often more useful than simply printing it or using `typeof()` or `class`, especially for beginners. Rather than showing every value, `str()` reports the object's overall type and class, its size or dimensions (e.g., the number of rows and columns in a data frame), and the internal structure of its components. For a data frame, str() lists each variable (column), shows its data type (numeric, character, factor, etc.), and prints a small preview of its values; for lists, it shows the elements and their types; and for more complex objects like model results, it reveals the named pieces that store coefficients, residuals, and other outputs. In practice, `str()` is a fast "diagnostic" function: it helps you confirm that your data imported correctly, spot variables that were read in as the wrong type (e.g., numbers stored as text), and understand what parts an object contains so you know how to access and use it.

Here are examples using objects from earlier sections:

```r
#Quick "is it numeric/character/logical?" checks
typeof(ages)
```

```
## [1] "double"
```

```r
class(ages)
```

```
## [1] "numeric"
```

```r
str(ages)
```

```
##  num [1:6] 25 31 19 42 50 22
```

```r
typeof(student_data)
```

```
## [1] "list"
```

```r
class(student_data)
```

```
## [1] "data.frame"
```

```r
str(student_data)
```

```
## 'data.frame':    4 obs. of  4 variables:
##  $ ID   : num  101 102 103 104
##  $ Name : chr  "Alice" "Bob" "Charlie" "David"
##  $ Score: num  88 92 75 85
##  $ Major: chr  "Biology" "History" "Math" "Biology"
```

R also provides a family of `is.*()` functions that return TRUE/FALSE and are very useful when you are debugging or cleaning data:

```r
is.numeric(ages)
```

```
## [1] TRUE
```

```r
is.character(name)
```

```
## [1] TRUE
```

```r
is.logical(is_complete)
```

```
## [1] TRUE
```

```r
is.data.frame(student_data)
```

```
## [1] TRUE
```

Rule of thumb for beginners:

- Use str() when you want the most helpful overview.
- Use class() when you care about how an object behaves (especially with data frames, tibbles, models).
- Use typeof() when you are focused on the underlying atomic type.

Getting help in R: `help()` and ?

R has a built-in help system that is essential when you are learning (and it remains essential even for experienced users). There are two common ways to open documentation for a function:

1. `help(function_name)`
2. `?function_name` (a shortcut for help())

For example:

```r
help(mean)
?mean
help(head)
?head
```

What to look for in a help page

Most help pages follow a standard structure. The sections students should focus on first are:

- Description: what the function is for
- Usage: the function call "template" (shows arguments and defaults)
- Arguments: what each input means
- Value: what the function returns
- Examples: runnable examples (often the fastest way to learn)

Finding functions when you do not know the name

If you know what you want to do but not the exact function name, you can search the help system using:

- `help.search("keyword")` or
- `??keyword` (shortcut for help.search())

Examples:

```
help.search("correlation")
??correlation
```

A practical next step after finding a candidate function is to read its examples and then try it on your own objects.